SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

Electronics Store Inventory Management System (ESIMS)

Team 8
Samiksha B N – PES1UG24CS413
Roopa Sreedhar A – PES1UG24CS386
Rishitth G R – PES1UG24CS376
Sakthevel Saravanan – PES1UG24CS405

Submission Contents

Part I – Software Requirements Specification (SRS)

Completed project requirements, use case model, functional/non-functional/security requirements, validation planning, project plan, field layouts and RTM.

Part II – Software Architecture & Design Specification (SAD)

Layered architecture, component diagram, module decomposition, class/deployment diagrams, three sequence diagrams, API/service contracts, security architecture, data integrity and traceability.

Part III – Software Test Plan (STP)

Testing strategy, security validation, environment, schedule, responsibilities, risks, 27 planned test cases, non-functional/security coverage and requirements traceability.

Software Requirements Specification

Electronics Store Inventory Management System

Field	Entry
Project Title	Electronics Store Inventory Management System
Group Number	Team 8
Project ID	Team 8
Technology Stack	Python (Django), PostgreSQL
Development Methodology	Agile
Version	1.0 (Approved)
Date Created	6/09/2026

Team details:

1) Samiksha B N – PES1UG24CS413
2) Roopa Sreedhar A- PES1UG24CS386
3) Rishitth G R – PES1UG24CS376
4) Sakthevel Saravanan – PES1UG24CS405

Table of Contents

Table of Contents.....2
Revision History.....4
1. Introduction.....4
1.1 Purpose.....4
1.2 Intended Audience and Reading Suggestions.....4
1.3 Product Scope.....4
1.4 References.....5
2. Overall Description.....5
2.1 Product Perspective.....5

2.2 Product Functions.....5
2.3 User Classes and Characteristics.....5
2.4 Operating Environment.....6
2.5 Design and Implementation Constraints.....6
2.6 Assumptions and Dependencies.....6
3. External Interface Requirements.....6
3.1 User Interfaces.....6
3.2 Software Interfaces.....6
3.3 Communications Interfaces.....7
4. Analysis Models.....7
4.1 Validation Specification – Initial Planning.....8
4.2 Project Plan.....9
4.3 Tool Stack.....9
5. System Features.....10
5.1 User Authentication & Role Management (Owner: Member 1).....10
5.1.1 Description and Priority.....10
5.1.2 Stimulus/Response Sequences.....10
5.1.3 Functional Requirements.....10
5.2 Product & Inventory Catalog Management (Owner: Member 2).....10
5.2.1 Description and Priority.....10
5.2.2 Stimulus/Response Sequences.....10
5.2.3 Functional Requirements.....10
5.3 Purchase & Sales Order Handling – Stock In/Out (Owner: Member 3).....11

5.3.1 Description and Priority.....11
5.3.2 Stimulus/Response Sequences.....11
5.3.3 Functional Requirements.....11
5.4 Supplier Management & Reporting (Owner: Member 4).....12
5.4.1 Description and Priority.....12
5.4.2 Stimulus/Response Sequences.....12
5.4.3 Functional Requirements.....12
6. Other Nonfunctional Requirements.....12
6.1 Performance Requirements.....12
6.2 Safety Requirements.....13
6.3 Security Requirements.....13
6.4 Software Quality Attributes.....13
6.5 Business Rules.....13
7. Other Requirements.....14
Appendix A: Glossary.....14
Appendix B: Field Layouts.....14 Appendix C: Requirement Traceability Matrix.....15

Revision History

Name	Date	Reason For Changes	Version
Team 8	6/09/2026	Initial draft	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Electronics Store Inventory Management System (ESIMS), Version 1.0. The system digitizes stock tracking, purchase and sales recording, supplier management and low-stock alerting for a retail electronics store. This document covers the complete system as a single deliverable; no subsystem is scoped separately.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team (all four members), the course faculty/evaluators, and any reviewer assessing the project's requirements coverage. Developers should read Sections 3–6 closely before implementation; evaluators may use Section 5 (System Features) and Appendix C (Traceability Matrix) to verify coverage. The document is organized following the standard SRS structure: overall description, interfaces, analysis models, system features, non-functional requirements, and appendices.

1.3 Product Scope

Electronics stores handle a large number of SKUs across categories such as mobiles, accessories, appliances and components, and struggle to track stock levels manually. This leads to overselling, stockouts, delayed reordering, and no visibility into which products are moving fastest. ESIMS provides a digital platform for managing the product catalog, recording purchase (stock-in) and sales (stock-out) transactions, tracking supplier information, generating low-stock alerts, and producing basic operational reports for store staff and administrators.

In Scope / Out of Scope

In Scope	Out of Scope
Staff / Admin login and role-based access	Barcode / QR hardware scanner integration
Product and category management	Multi-store / warehouse transfer management
Stock-in (purchase) recording	Point-of-sale hardware integration
Stock-out (sales) recording	Online customer-facing storefront
Low-stock threshold alerts	Payment gateway integration
Supplier management	Accounting / ERP system integration
Inventory and sales reporting	AI-based demand forecasting

In Scope	Out of Scope
Search and filter inventory	Multi-currency / multi-branch enterprise deployment

1.4 References

- PES University – Jackfruit Mini-Project: Software Engineering Project Guidelines 2026 (UE24CS341A).
- PES University – SE Level 3 Mini-Project Student Submission Guide (Coffee Kiosk Exemplar).
- Approved Project Synopsis – Electronics Store Inventory Management System, v1.0.

2. Overall Description

2.1 Product Perspective

ESIMS is a new, self-contained web application; it does not replace an existing system. It is a single Django-based application with a server-rendered (or API-backed) frontend, a relational database, and role-based access for two internal user classes: Staff and Administrator. The system does not integrate with any external e-commerce or payment platform in this version.

2.2 Product Functions

- User authentication and role-based access control (Staff / Administrator).
- Product and category catalog management (create, edit, disable).
- Stock-in (purchase entry) and stock-out (sales entry) transaction recording.
- Automatic low-stock alert generation based on a configurable reorder threshold.
- Supplier record management.
- Inventory and sales reporting (stock value, top-selling items, low-stock list).
- Search and filter across the product catalog.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Staff	Login; browse/search catalog; record stock-in and stock-out transactions; view low-stock alerts	Non-technical to moderately technical; frequent daily use during business hours
Administrator	All Staff privileges, plus: manage products/categories, manage suppliers, manage staff accounts, view reports	Higher privilege; trained user; less frequent but higher-impact use

2.4 Operating Environment

The system runs as a Django web application deployed on a Linux server, backed by a PostgreSQL database. It is accessed through a standard modern web browser (Chrome, Edge, Firefox) on desktop devices used at the store counter/back office. No native mobile application is planned for this version.

2.5 Design and Implementation Constraints

- Must be implemented using Python (Django) with a relational database (PostgreSQL/SQLite).
- Must follow Agile methodology with sprint-wise delivery, tracked via GitHub Projects.
- Must use Git/GitHub for version control with individual, attributable commits per member.
- Must include automated unit tests and a CI pipeline (GitHub Actions).
- Passwords must be hashed; plaintext storage of credentials is not permitted.

2.6 Assumptions and Dependencies

- It is assumed the store has a single physical location (no multi-branch stock transfer in v1.0).
- It is assumed Staff and Administrator accounts are created internally; there is no public self-registration.
- The system depends on Django's built-in authentication framework for session/user management.
- It is assumed reorder thresholds are set per product by an Administrator and do not change automatically.

3. External Interface Requirements

3.1 User Interfaces

The system provides a web-based interface with two role-driven views: a Staff view (catalog browsing, stock-in/stock-out forms, alerts) and an Administrator view (all Staff screens plus product/category/supplier/user management and reports). Standard elements include a persistent navigation bar, a dashboard summarizing low-stock items, form validation messages, and confirmation dialogs for destructive actions (e.g., disabling a product).

3.2 Software Interfaces

Component	Version / Type	Purpose
Django	5.x	Backend web framework, ORM, authentication
PostgreSQL	15.x	Primary relational data store
Django Templates + Bootstrap	Bootstrap 5	Server-rendered UI
Django REST Framework (optional)	3.x	API layer if a decoupled frontend is used

3.3 Communications Interfaces

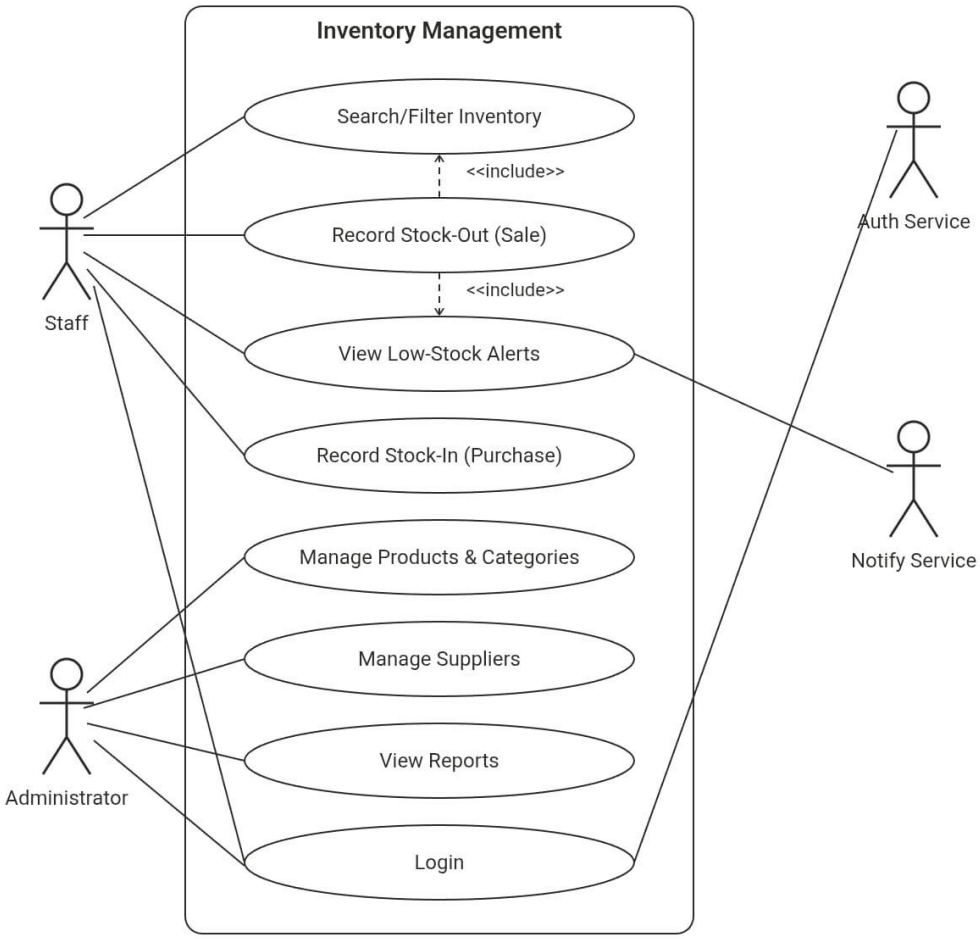
All client-server communication occurs over HTTPS. The system uses standard HTTP request/response cycles for form submissions and page loads; if a REST API is used, JSON is the data-interchange format. No email/SMS notification channel is included in this version; low-stock alerts are surfaced in-app only.

4. Analysis Models

The primary analysis model at this stage is the use case model below, covering all actors and major interactions. An Entity-Relationship model (Product, Category, Supplier, StockTransaction, User) will be developed in the Design phase and referenced from the Requirement Traceability Matrix (Appendix C).

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Login	Staff / Admin	Valid account exists	Authenticated session
UC-02	Manage Product Catalog	Administrator	Admin authenticated	Product created/updated/disabled
UC-03	Manage Categories	Administrator	Admin authenticated	Category created/updated
UC-04	Record Stock-In (Purchase)	Staff	Logged in; valid supplier	Stock quantity increased
UC-05	Record Stock-Out (Sale)	Staff	Logged in; sufficient stock	Stock quantity decreased
UC-06	Search / Browse Inventory	Staff / Admin	Logged in	Matching products displayed
UC-07	View Low-Stock Alerts	Staff / Admin	Logged in	Alert list displayed
UC-08	Manage Suppliers	Administrator	Admin authenticated	Supplier created/updated/removed
UC-09	Generate Reports	Administrator	Admin authenticated	Report displayed/exported
UC-10	Manage Staff Accounts	Administrator	Admin authenticated	Staff account created/deactivated

Use Case Diagram



4.1 Validation Specification – Initial Planning

At the requirements stage, the validation specification is planning-focused. It explains how functional, security and non-functional requirements will eventually be validated, including planned penetration/security testing. This stage is not the final test report.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Valid/invalid stock-out transaction
Security	Authentication/authorization + security testing	Staff account attempts an admin-only endpoint
Performance	Load/response-time test	Concurrent catalog search requests

Usability	Task-based walkthrough	Staff completes a stock-in transaction in ≤4 steps
Requirement Category	**Planned Validation**	**Example**
Reliability	Failure/retry scenarios	Duplicate stock-out on form doublesubmission
Unit	Automated unit tests + coverage	Stock quantity and low-stock threshold calculation

4.2 Project Plan

The project plan identifies and justifies the lifecycle, identifies tools, classifies reuse/build components, and breaks work down into a detailed work breakdown structure (WBS) with effort estimates per member.

Work Package	Tasks	Owner	Estimated Effort (person-days)	Output
WP1	Requirements + use cases	All	4	SRS
WP2	Architecture + UML	M1/M2	4	Architecture/Design
WP3	Auth & Role Management module	M1	6	Working feature
WP4	Product & Inventory Catalog module	M2	6	Working feature
WP5	Stock In/Out module	M3	7	Working feature
WP6	Supplier Management & Reporting module	M4	7	Working feature
WP7	Testing + CI/CD + security	All	6	Test/validation evidence
WP8	Final validation + report	All	4	Final report

4.3 Tool Stack

The following tools are used across the project lifecycle for version control, Agile planning, CI/CD, containerization, testing and design.

Activity	Recommended/Example Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	Jira / GitHub Projects	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Pipeline screenshot/log
Containerization	Docker	Dockerfile + build/run evidence
Testing	Django test framework / pytest	Test results
Static analysis	SonarQube or similar	Analysis report

API testing	Postman / similar	Collection/results
Design	Draw.io / StarUML / PlantUML	UML diagrams

5. System Features

Features are organized by module, each corresponding to one team member's functional ownership as defined in the project Synopsis.

5.1 User Authentication & Role Management (Owner: Member 1- Roopa Sreedhar A)

5.1.1 Description and Priority

Provides secure login and role-based access control distinguishing Staff from Administrator users. Priority: High.

5.1.2 Stimulus/Response Sequences

User submits login credentials → system validates and creates a session, or returns an error. User navigates to a restricted page → system checks role and grants or denies access.

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a user to log in using a unique username/email and password.	High	Functional test
FR-002	The system shall restrict access to each feature based on the user's assigned role (Staff or Administrator).	High	Authorization test
FR-003	The system shall allow an Administrator to create, edit, and deactivate Staff accounts.	High	Functional test
FR-004	The system shall terminate a user's session after a configurable period of inactivity.	Medium	Security test

5.2 Product & Inventory Catalog Management (Owner: Member 2- Sakthevel Saravanan)

5.2.1 Description and Priority

Allows authorized users to maintain the product catalog and view live stock levels. Priority: High.

5.2.2 Stimulus/Response Sequences

Admin submits a new product form → system validates and creates the product record. User enters a search term → system returns matching products with current stock.

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification

FR-005	The system shall allow an authorized user to add a new product with name, category, SKU, price and initial stock quantity.	High	Functional test
FR-006	The system shall allow an authorized user to edit product details including price and category.	High	Functional test
FR-007	The system shall allow an Administrator to disable a product without deleting its historical transaction records.	Medium	Functional test
FR-008	The system shall allow users to search and filter products by name, category, or SKU.	High	Functional test
FR-009	The system shall display the current stock quantity for each product in real time.	High	System test

5.3 Purchase & Sales Order Handling – Stock In/Out (Owner: Member 3 – Samiksha B N)

5.3.1 Description and Priority

Core transactional module: records purchases (stock-in) and sales (stock-out), keeping stock counts accurate and consistent. Priority: High.

5.3.2 Stimulus/Response Sequences

Staff submits a stock-in form against a supplier → system increases stock and logs the transaction. Staff submits a stock-out form → system validates available quantity, decreases stock, and logs the transaction, or rejects it if insufficient stock is available.

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-010	The system shall allow a Staff member to record a stock-in (purchase) transaction against a registered supplier, increasing product stock.	High	Functional test
FR-011	The system shall allow a Staff member to record a stock-out (sale) transaction, decreasing product stock.	High	Functional test
FR-012	The system shall reject a stock-out transaction that would reduce stock below zero.	High	Negative test
FR-013	The system shall assign a unique transaction ID to every stockin and stock-out record.	High	Integration test
FR-014	The system shall maintain an audit trail (user, timestamp, quantity) for every stock transaction.	High	System test

5.4 Supplier Management & Reporting (Owner: Member 4- Rishitth G R)

5.4.1 Description and Priority

Maintains supplier records, generates low-stock alerts, and produces operational reports for the Administrator. Priority: Medium-High.

5.4.2 Stimulus/Response Sequences

Stock level drops below reorder threshold → system generates a low-stock alert automatically. Admin requests a report for a date range → system computes and displays aggregated stock/sales data.

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-015	The system shall allow an Administrator to add, edit, and remove supplier records.	Medium	Functional test
FR-016	The system shall associate each stock-in transaction with a registered supplier.	High	Integration test
FR-017	The system shall generate a low-stock alert when a product's quantity falls below its configured reorder threshold.	High	System test
FR-018	The system shall generate a stock/sales report for a specified date range.	Medium	Functional test
FR-019	The system shall display the top-selling products based on recorded sales transactions.	Low	Functional test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of catalog search requests shall receive a response within 2 seconds under normal load.	Performance test
NFR-002	Availability	The application shall be available at least 99% during defined business hours, excluding planned maintenance.	Operational review
NFR-003	Usability	A Staff member shall be able to complete a stock-in or stockout transaction in no more than 4 screens/steps.	Usability walkthrough
NFR-004	Reliability	A stock transaction shall not be recorded twice if a user double-submits the same form.	Negative/ integration test
NFR-005	Maintainability	Business logic shall be separated into distinct Django apps/modules with automated unit tests.	Code review
ID	**Category**	**Measurable Requirement**	**Verification**

NFR-006	Testability	Core services (stock calculation, alert generation) shall be testable independent of the UI layer.	Unit/integration test

6.2 Safety Requirements

As ESIMS is an internal business application with no physical or life-safety impact, no dedicated safety certifications apply. The primary safety-adjacent concern is data integrity: the system shall never allow stock counts to become negative or transactions to be silently lost.

6.3 Security Requirements

Security Objectives:SO-01 Protect staff credentials, inventory, supplier and pricing information from unauthorized access; SO-02 preserve integrity and accountability of stock transactions and securityrelevant actions through validation, RBAC, audit logging and controlled sessions.

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall not be stored in plaintext (hashed using Django's default password hasher).	Code/configuration review
SEC-002	Role-based access shall prevent Staff accounts from accessing Administrator-only functions.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied input before processing (e.g., stock quantity fields).	Negative/security tests
SEC-004	Authenticated sessions shall expire after a defined period of inactivity.	Security test
SEC-005	Sensitive supplier and pricing data shall not be exposed to unauthenticated users.	Design review
SEC-006	Security-relevant events (failed logins, unauthorized access attempts) shall be logged.	System test

6.4 Software Quality Attributes

- Maintainability: modules (auth, catalog, transactions, suppliers/reports) shall be loosely coupled Django apps.
- Reusability: stock calculation logic shall be implemented as reusable service functions, not duplicated across views.
- Robustness: the system shall handle invalid or missing form input gracefully with clear error messages.
- Usability: the interface shall favor consistency – the same layout and action patterns across catalog, stock, and supplier screens.

6.5 Business Rules

- Only products marked "active" can be included in a new stock-out (sale) transaction.
- A stock-out transaction cannot exceed the currently available stock quantity for a product.
- Only Administrators can add, edit, or remove suppliers and categories.

- Only Administrators can deactivate a product or a Staff account.
- Every stock transaction must be linked to a valid, active product.
- A low-stock alert is generated automatically and cannot be manually suppressed by Staff.

7. Other Requirements

- Database requirements: all stock transactions must be retained indefinitely for audit purposes (no hard deletes); products are soft-deleted (disabled) rather than removed.
- Legal/compliance: no personally identifiable customer data is collected in this version, limiting data-protection scope to staff account credentials.
- Reuse objectives: Django's built-in admin, auth, and ORM are reused wherever possible instead of custom-built equivalents.
- Localization: currency is assumed to be INR (₹) for this version; no multi-currency support is required.

Appendix A: Glossary

Term	Definition
SKU	Stock Keeping Unit – a unique identifier for a distinct product/variant.
Stock-in	A transaction that increases a product's recorded stock quantity (a purchase from a supplier).
Stock-out	A transaction that decreases a product's recorded stock quantity (a sale).
Reorder threshold	The minimum stock quantity below which a low-stock alert is triggered.
RBAC	Role-Based Access Control – restricting system functions based on a user's assigned role.
Administrator	A user role with full privileges, including catalog, supplier and staff-account management.
Staff	A user role limited to day-to-day catalog viewing and stock transaction recording.

Appendix B: Field Layouts

Sample field layout for the Product entity:

Field	Length	Data Type	Description	Mandatory
product_id	36	UUID/String	Unique product identifier	Y
sku	50	String	Stock Keeping Unit code	Y
product_name	150	String	Display name of the product	Y
Field	**Length**	**Data Type**	**Description**	**Mandatory**

category_id	36	UUID/String	Reference to product category	Y
price	10,2	Decimal	Current selling price	Y
quantity_in_stock	10	Integer	Current available stock quantity	Y
reorder_threshold	10	Integer	Minimum quantity before a low-stock alert fires	Y
supplier_id	36	UUID/String	Reference to the associated supplier	N
status	20	Enum/String	active / disabled	Y

Sample report field requirements:

Stock Report	Sales Report
Product Name	Transaction Reference Number
SKU	Product Name
Category	Quantity Sold
Current Stock	Sale Amount
Reorder Threshold	Transaction Date
Stock Value (Qty × Price)	Staff Member

Appendix C: Requirement Traceability Matrix

Architecture and design references will be finalized during the Design phase (Deliverable 3); the entries below show the planned mapping to be completed once modules are implemented.

Req ID	Brief Description	Architecture Ref	Design Ref	Code/File Ref	Test Case ID	System Test ID
FR-001	User login	AUTH-01	SEQ-01 / Login	auth/views.py	UT-AUTH-01	ST-AUTH-01
FR-005	Add product	CATALOG-01	COMP-02 / Product Management	catalog/services.py	UT-CAT-01	ST-CAT-01
FR-010	Record stock-in	STOCK-01	SEQ-02 / Stock-In	stock/services.py	IT-STOCK-01	ST-STOCK-01
FR-012	Reject over-sale	STOCK-02	SEQ-02 / Stock Validation	stock/services.py	UT-STOCK-02	ST-STOCK-02
FR-017	Low-stock alert	REPORT-01	SEQ-03 / Low-Stock Alert	reports/alerts.py	UT-ALERT-01	ST-ALERT-01
SEC-002	Role-based access	AUTH-02	COMP-01 / RBAC	middleware/auth.py	UT-SEC-01	ST-SEC-01

Req ID	Brief Description	Architecture Ref	Design Ref	Code/File Ref	Test Case ID	System Test ID
NFR-001	Search response time	API-01	API-02 / Search Contract	catalog/views.py	PERF-01	ST-PERF-01

Software Architecture and Design Specification

Electronics Store Inventory Management System (ESIMS)

Project	Electronics Store Inventory Management System (ESIMS)
Group / Project ID	Team 8
Version	1.0
Date	30-09-2026
Technology	Python (Django), PostgreSQL
Methodology	Agile
Authors	Team 8
Status	Draft

Revision History

Version	Date	Author	Change Summary
1.0	30-09-2026	Team 8	Architecture, UML, API, security and design completed from approved SRS.

Approvals

Role	Name	Signature / Date
Team / Faculty Reviewer	Team 8	

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification defines the architecture, major components, interfaces, data design, security architecture and detailed interaction design for the Electronics Store Inventory Management System (ESIMS), based on the approved SRS.

1.2 Scope

The design covers authentication and role management, product/category catalog management, stock-in and stock-out transactions, supplier management, low-stock alerts, inventory/sales reporting and inventory search/filtering. It does not introduce the out-of-scope integrations listed in the SRS.

1.3 Audience

Developers, testers, faculty/evaluators, maintainers and security reviewers.

1.4 Definitions

- ESIMS – Electronics Store Inventory Management System
- RBAC – Role-Based Access Control
- SKU – Stock Keeping Unit
- Stock-in – purchase transaction increasing inventory
- Stock-out – sales transaction decreasing inventory
- NFR – Non-Functional Requirement
- RTM – Requirements Traceability Matrix.

2. Document Overview

2.1 How to Use this Document

Sections 3 and 4 describe the architecture and detailed design respectively. The diagrams show component responsibilities, domain relationships, runtime deployment and key request flows. Traceability connects the design back to SRS requirements.

2.2 Related Documents

- ESIMS Software Requirements Specification, Version 1.0 (approved).
- ESIMS Software Test Plan, Version 1.0.
- PES University Software Engineering Level 3 Student Submission Guide / Coffee Kiosk exemplar.

3. Architecture

3.1 Goals & Constraints

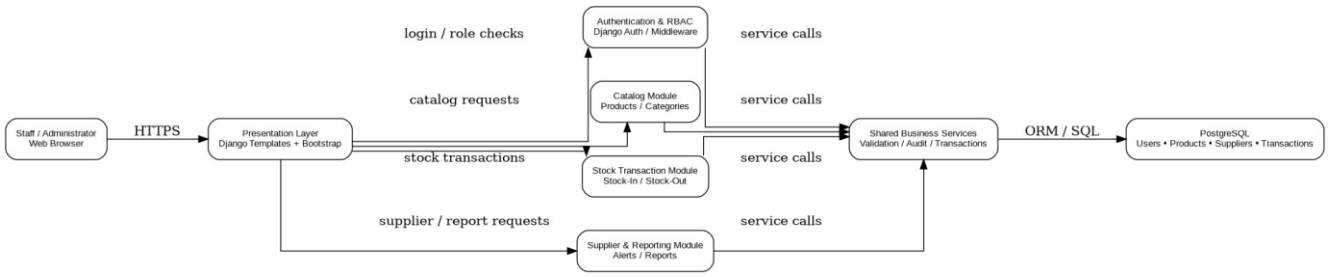
- Maintain accurate and auditable stock quantities and prevent negative stock.

- Enforce Staff/Administrator role separation and protect credentials and sensitive data.
- Support the SRS performance target: 95% of catalog searches within 2 seconds under normal load.
- Support at least 99% availability during defined business hours, excluding planned maintenance.
- Keep stock-in/stock-out workflows within four screens/steps for Staff.
- Use Python/Django with PostgreSQL, Git/GitHub, automated tests and CI as specified by the SRS.
- Single-store deployment and no public self-registration are constraints for Version 1.0.

3.2 Stakeholders & Concerns

Stakeholder	Primary architectural concern
Staff	Fast, clear catalog and stock transaction workflows; access only to permitted functions.
Administrator	Complete management of products, categories, suppliers, staff accounts and reports.
Developers	Modularity, testability, maintainability and clear separation of concerns.
Faculty / Evaluators	Requirement coverage, UML/design completeness, traceability and security validation.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Responsibilities	Key interfaces	Requirements
Presentation Layer	Server-rendered pages/forms, navigation, validation messages and role-driven views.	HTTPS requests; Django views/templates	FR-001–FR-019, NFR-003
Authentication & RBAC	Login, password verification, session lifecycle and role checks.	Login service; authorization middleware	FR-001–FR-004, SEC-001/002/004
Catalog Module	Product/category CRUD, search/filter, active/disabled status and stock display.	Product/category services	FR-005–FR-009
Stock Transaction Module	Atomic stock-in/out, quantity validation, transaction IDs and audit records.	Stock service; transaction repository	FR-010–FR-014, NFR-004
Supplier & Reporting Module	Supplier CRUD, low-stock alerts and operational reports.	Supplier/report/alert services	FR-015–FR-019
PostgreSQL Data Layer	Persistent users, products, categories, suppliers, transactions and audit/alert data.	Django ORM / SQL	NFR-005/006, SEC-005/006

3.5 Chosen Architecture Pattern and Rationale

A layered architecture with modular Django applications is selected. The presentation layer handles HTTP/UI concerns, authentication/RBAC handles identity and authorization, feature modules implement business rules, shared services enforce reusable transaction and validation logic and the data layer isolates PostgreSQL persistence.

A microservices architecture is not selected for Version 1.0 because the SRS describes a single-location, self-contained application with no external e-commerce/payment integrations. Additional service decomposition can be considered if deployment scale or integration scope changes.

3.6 Technology Stack & Data Stores

Layer	Technology	Purpose
Web application	Python, Django 5.x	Routing, views, ORM, authentication and business application.
UI	Django Templates + Bootstrap 5	Server-rendered, role-driven web interface.
Database	PostgreSQL 15.x	Relational persistence and transaction integrity.
Optional API	Django REST Framework 3.x	API layer if a decoupled frontend is used.
Testing / CI	Django test framework or pytest; GitHub Actions	Automated tests and continuous integration.
Design / API tools	Draw.io / PlantUML; Postman	Architecture/UML diagrams and API validation.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Concurrent stock updates	Incorrect inventory quantity or duplicate transactions.	Use database transactions/atomic updates, validation and unique transaction IDs; test double submission.
Unauthorized admin access	Exposure or modification of sensitive data.	Central RBAC middleware, server-side authorization tests and least-privilege checks.
Database/server outage	Unavailable inventory functions.	Operational monitoring, database backup procedure and clear recovery steps.
Slow catalog search	Failure of NFR-001.	Indexed search fields, query review and JMeter/load testing.
Invalid input	Data corruption or application errors.	Server-side validation, sanitization and negative tests.

3.8 Traceability to Requirements

SRS requirement	Architecture ref	Design element	Test reference
FR-001 Login	AUTH-01	SEQ-01; Auth component	UT-AUTH-01 / ST-AUTH-01
FR-005 Add product	CATALOG-01	Product service; Class Product	UT-CAT-01 / ST-CAT-01
FR-010 Stock-in	STOCK-01	Stock service; transaction model	IT-STOCK-01 / ST-STOCK-01
FR-012 Reject over-sale	STOCK-02	Stock validation rule	UT-STOCK-02 / STSTOCK-02
FR-017 Low-stock alert	REPORT-01	Alert service; SEQ-03	UT-ALERT-01 / ST-ALERT-01
SEC-002 RBAC	AUTH-02	Authorization middleware	UT-SEC-01 / ST-SEC-01
NFR-001 Search response	API-01	Search API/service design	PERF-01 / ST-PERF-01

3.9 Security Architecture

Security objectives are to protect credentials and sensitive internal data and to preserve the integrity/accountability of stock transactions. The architecture applies defense in depth across authentication, authorization, transport, validation, persistence and audit logging.

STRIDE concern	ESIMS control	Relevant SRS requirements
Spoofing	Django authentication, password hashing, session expiry	SEC-001, SEC-004
Tampering	Server-side validation, CSRF protection, atomic stock updates, database constraints	SEC-003, FR-012, NFR-004
Repudiation	Audit trail containing user, timestamp and quantity; security-event logging	FR-014, SEC-006
Information disclosure	HTTPS, authorization checks, protected supplier/pricing data and non-sensitive logs	SEC-005
Denial of service	Input limits/validation; rate limiting for externally exposed APIs; load testing	NFR-001
Elevation of privilege	Central RBAC middleware and adminonly views/actions	FR-002, SEC-002

4. Design

4.1 Design Overview

The design decomposes ESIMS into independently testable Django modules while keeping shared business rules in service functions. Product, Category, Supplier, User and StockTransaction are the principal domain entities. Stockchanging operations are treated as atomic transactions so that inventory quantity and transaction history remain consistent.

4.1.1 Module Decomposition

Module	Responsibilities	Dependencies	Owner / functional area
Authentication	Login, roles, session expiry, staff-account management	Django Auth, User	Roopa Sreedhar A
Catalog	Products, categories, search/filter, active status	Product/Category models	Sakthevel Saravanan
Stock	Stock-in/out, validation, IDs, audit trail	Product, Supplier, User, Transaction	Samiksha B N
Supplier & Reporting	Supplier CRUD, alerts, stock/sales reports	Supplier, Product, Transaction	Rishitth G R

4.1.2 Domain / Class Diagram

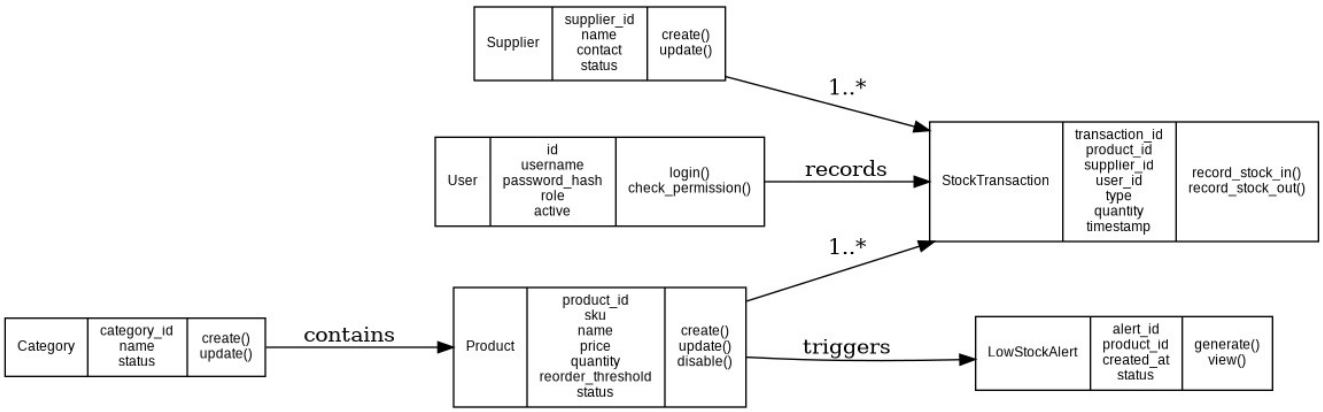


Figure 1. Major ESIMS domain classes and relationships.

4.2 UML Sequence Diagrams

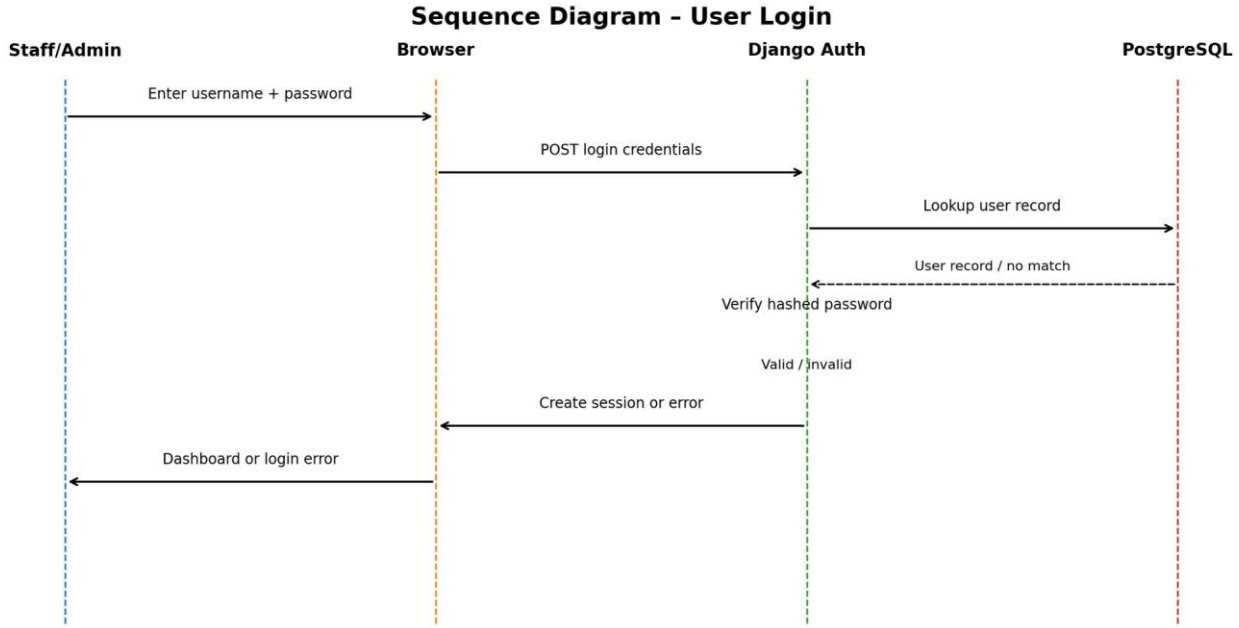


Figure 2. Login flow.

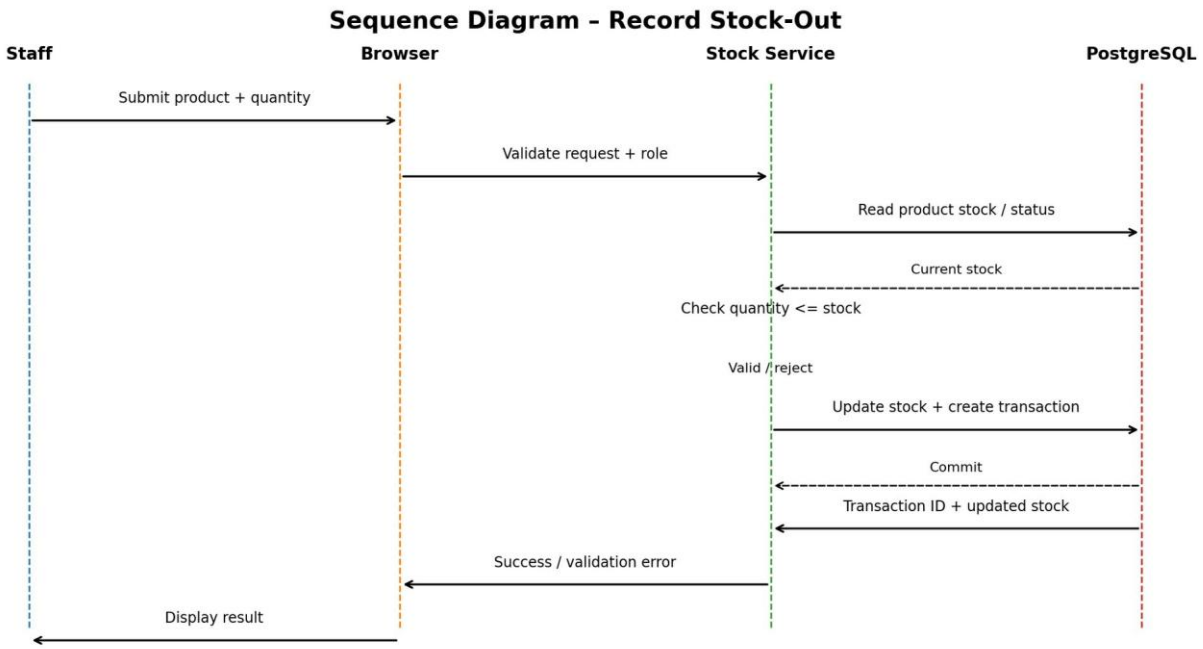


Figure 3. Stock-out flow.

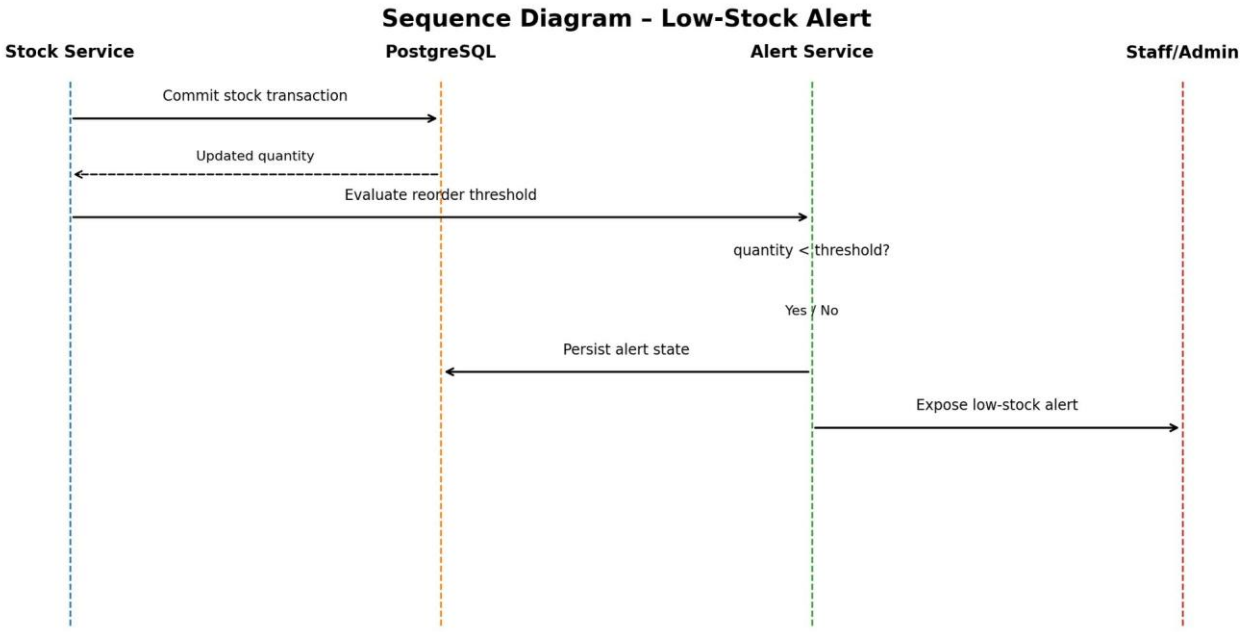


Figure 4. Low-stock alert flow.

4.3 API / Service Interface Design

The SRS permits an optional Django REST Framework API when a decoupled frontend is used. The following logical contracts define stable interfaces for the core operations. The same contracts can be implemented as Django service functions in a server-rendered deployment.

Method	Endpoint / Service	Purpose	Input	Success	Errors / Security
POST	/api/auth/login	Authenticate user	username/email, password	Session/token + role	401 invalid credentials; rate limiting; no
					password logging
GET	/api/products	Search/filter catalog	name/category/ SKU query	Product list + current stock	401 unauthenticated; validation of query parameters
POST	/api/stock-in	Record purchase	product_id, supplier_id, quantity	Transaction ID + new stock	400 validation; 403 role; 404 missing product/supplier; 409 conflict
POST	/api/stock-out	Record sale	product_id, quantity	Transaction ID + new stock	400 validation; 403 role; 409 insufficient stock
GET	/api/alerts/lowstock	View low-stock items	Authenticated user	Alert list	401 unauthenticated
GET	/api/reports/sales	Generate sales report	from_date, to_date	Aggregated report	403 admin-only; 400 invalid dates

4.4 Error Handling, Logging & Monitoring

- Validation errors return clear field-level messages without exposing implementation details.
- Authentication failures use generic messages so that account existence is not unnecessarily disclosed.
- Authorization failures are distinguished from authentication failures internally and are logged as security events.
- Stock operations use database transactions and roll back on failure so quantity and transaction history cannot diverge.
- Duplicate submissions are detected through transaction/idempotency safeguards and database uniqueness.
- Logs record timestamp, event type, user identifier where appropriate, request outcome and correlation information
- Passwords and other sensitive credentials are never logged.
- Operational monitoring tracks application availability, search response time, transaction failures, failed logins and unauthorized access attempts.

4.5 UX Design

- Persistent navigation differs by role: Staff sees catalog, stock operations and alerts
- Administrators additionally see product/category, supplier, staff-account and reporting functions.
- Dashboards surface current low-stock items and important operational messages.
- Forms use validation messages and confirmation dialogs for destructive actions such as disabling products.
- Stock-in and stock-out flows are designed to fit within the SRS limit of four screens/steps.
- Consistent Bootstrap layouts and terminology are used across catalog, stock and supplier screens.

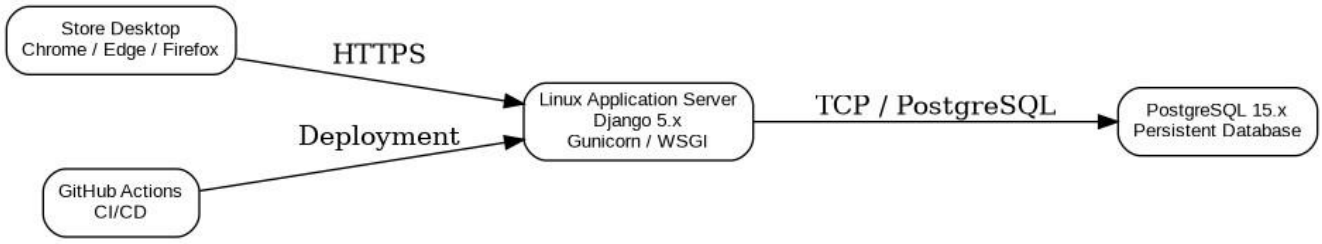
4.6 Database / Data Integrity Design

The relational model centers on Product, Category, Supplier, User and StockTransaction. Products are disabled rather than hard-deleted, while stock transactions are retained indefinitely for audit. Every stock transaction must be assigned a unique transaction ID and be linked to an active product in the system. When processing a stock-in transaction, a valid supplier must be associated with the record. Conversely, for a stock-out transaction, the requested quantity cannot exceed the current available stock for that product.

Entity	Key fields	Relationships	Integrity rule	Retention
User	user_id, username, role, password_hash	records StockTransaction	unique login; role constrained	Account lifecycle
Category	category_id, name, status	1-to-many Product	unique/valid category	Persistent
Product	product_id, SKU, price, quantity, threshold, status	belongs to Category; has Transactions/Alerts	quantity >= 0; disabled products cannot be sold	Soft delete

Supplier	supplier_id, name, contact, status	associated with StockTransaction	stock-in requires valid supplier	Persistent
StockTransaction	transaction_id, product, supplier, user, type, quantity, timestamp	references Product/User/Supplier	unique ID; quantity > 0; immutable history	Indefinite

4.7 Deployment Design



4.8 Open Issues & Next Steps

- Finalize exact API endpoints only if a decoupled frontend is selected; otherwise use the same service contracts internally.
- Add production deployment hardening, database backup/restore procedures and operational monitoring configuration.
- Perform security testing and performance testing against the implemented build before final sign-off.
- Future versions may consider multi-branch inventory, barcode/QR integration, POS integration or demand forecasting, all of which are out of scope for Version 1.0.

5. Appendices

5.1 Glossary

- RBAC – Role-Based Access Control.
- SKU – Stock Keeping Unit.
- Stock-in – purchase transaction increasing stock.
- Stock-out – sales transaction decreasing stock.
- RTM – Requirements Traceability Matrix.

5.2 References

- ESIMS SRS Team 8, Version 1.0.
- PES University Software Engineering Level 3 Student Submission Guide / Coffee Kiosk exemplar.
- PES University Software Engineering Mini-Project Deliverables Part-1.

5.3 Tools

- Draw.io / Graphviz / PlantUML for architecture and UML diagrams.
- Postman for API testing if REST endpoints are enabled.
- Django test framework / pytest and GitHub Actions for automated testing and CI.

Software Test Plan

Electronics Store Inventory Management System (ESIMS)

Project	Electronics Store Inventory Management System (ESIMS)
Group / Project ID	Team 8
Version	1.0
Date	30-09-2026
Technology	Python (Django), PostgreSQL
Methodology	Agile
Authors	Team 8
Status	Sample/Draft

Revision History

Version	Date	Author	Change Summary
1.0	30-09-2026	Team 8	Completed functional, nonfunctional, security and traceability planning.

1. Introduction

Purpose:

This Software Test Plan defines the testing objectives, scope, strategy, environment, test cases, security validation, traceability and reporting approach for ESIMS Version 1.0.

Scope:

Testing covers authentication/RBAC, product and category management, stock-in/out, supplier management, lowstock alerts, reporting, search/filtering and the stated non-functional and security requirements. Out-of-scope items from the SRS, such as barcode hardware, POS hardware, payment gateways, ERP integration and AI demand forecasting, are not tested.

References:

- ESIMS SRS v1.0; ESIMS SAD v1.0
- PES University Software Engineering Level 3 Student Submission Guide
- Mini-Project Deliverables Part-1.

Definitions:

- FR – Functional Requirement
- NFR – Non-Functional Requirement
- SEC – Security Requirement
- RTM – Requirements Traceability Matrix
- UAT – User Acceptance Testing.

2. Test Items

- Authentication and session management module.
- Role-based authorization and staff-account management.
- Product and category catalog module.
- Search/filter and current-stock display.
- Stock-in and stock-out transaction module.
- Supplier management module.
- Low-stock alert service.
- Inventory and sales reporting.
- Cross-cutting security, reliability, performance and usability behavior.

3. Features to be Tested

Requirement group	Feature	Primary validation
FR-001–FR-004	Login, RBAC, staff accounts, session expiry	Functional + authorization + security
FR-005–FR-009	Product/category management, search, current stock	Functional + system + performance

FR-010–FR-014	Stock-in/out, over-sale rejection, IDs, audit trail	Functional + integration + reliability
FR-015–FR-019	Suppliers, low-stock alerts, reports, top sellers	Functional + integration + system
NFR-001	95% catalog searches within 2 seconds	Performance/load test
NFR-002	99% business-hour availability	Operational review
NFR-003	Stock transaction within four screens/steps	Usability walkthrough
NFR-004	No duplicate transaction on double submission	Negative/integration test
NFR-005–NFR-006	Modularity and independent testability	Code review + unit/integration
SEC-001–SEC-006	Credential, authorization, validation, data exposure and logging controls	Security validation

4. Features Not to be Tested

- Barcode / QR hardware scanner integration.
- Point-of-sale hardware integration.
- Online customer-facing storefront.
- Payment gateway integration.
- Accounting / ERP integration.
- AI-based demand forecasting.
- Multi-store / warehouse transfer management.
- Multi-currency / multi-branch enterprise deployment.

5. Test Approach / Strategy

5.1 Test Levels

- Unit testing: service-level calculations, validation and alert generation independent of the UI.
- Integration testing: Django modules with PostgreSQL and cross-module transaction behavior.
- System testing: complete workflows through the web interface.
- Acceptance testing: representative Staff and Administrator tasks against SRS acceptance criteria.

5.2 Test Types

- Functional and negative testing.
- Regression testing after fixes or module changes.
- Performance/load testing for catalog search.
- Usability testing for stock transaction workflows.
- Reliability testing for duplicate submissions and transaction rollback.
- Security testing for authentication, RBAC, input validation, session expiry, sensitive-data exposure and security-event logging.

5.3 Entry and Exit Criteria

Entry criteria:

The test environment is fully prepared with a stable build, completed database schema migrations, representative test data, accessible systems, and all critical dependencies available.

Exit criteria:

All planned test cases have been executed, critical and high-severity blockers have been resolved or formally accepted, non-functional requirement thresholds and security-critical tests have been evaluated and completed, and the Requirements Traceability Matrix has been updated with execution evidence.

5.4 Test Automation Strategy

Use Django's test framework or pytest for unit/integration tests, Postman for API testing if REST is enabled, and JMeter or an equivalent load tool for the NFR-001 response-time test. CI through GitHub Actions should execute automated tests on pushes/pull requests.

5.5 Security Validation

Security objective / requirement	Validation activity	Expected evidence	Test ID
Protect credentials – SEC-001	Inspect password storage and authentication configuration.	Password hashes only; no plaintext credentials.	ST-SEC-01
Prevent unauthorized privilege – SEC-002	Attempt Staff access to Administrator-only functions.	Access denied and security event recorded.	ST-SEC-02
Validate/sanitize input – SEC-003	Submit invalid, negative, oversized and malformed values.	Request rejected without data corruption.	ST-SEC-03
Session protection – SEC-004	Wait/force inactivity timeout and reuse expired session.	Expired session requires reauthentication.	ST-SEC-04
Protect sensitive data – SEC-005	Access protected pages/API without authentication.	No sensitive supplier/pricing data disclosed.	ST-SEC-05
Security logging – SEC-006	Generate failed login and unauthorized access attempts.	Relevant events logged without passwords/secrets.	ST-SEC-06

6. Test Environment

- Server: Linux environment running Django 5.x application.
- Database: PostgreSQL 15.x.
- Clients: current Chrome, Edge and Firefox desktop browsers.
- Test tools: Django test framework / pytest, Postman if REST API is enabled, JMeter for performance, browser developer tools and GitHub Actions.
- Test data: dummy Staff/Admin accounts; active/disabled products; categories; registered suppliers; stock-in/out transactions; threshold boundary cases; duplicate submissions.
- Network: HTTPS client-server communication.

8. Test Schedule

Phase	Planned activity	Exit evidence
Test design	Derive test cases from FR/NFR/SEC requirements and SAD interfaces.	Approved test case set + RTM.
Environment preparation	Create test database, accounts, products, suppliers and baseline data.	Environment checklist.
Unit / integration	Execute service and persistence tests.	Automated test results.
System / security	Execute end-to-end workflows and security tests.	Execution log + defects.
Performance / usability	Measure search response and stock transaction steps.	Performance metrics + walkthrough notes.
Regression / UAT	Retest fixes and representative Staff/Admin workflows.	Final test summary and sign-off.

8. Test Deliverables

- Test Plan
- Test case specification
- Automated unit/integration test results
- Performance/security test evidence
- Defect reports
- Updated RTM
- Final Test Summary Report

9.Roles and Responsibilities

Role	Assigned to	Responsibility
QA / Test Coordination	Team 8 – shared	Maintain test plan, coordinate execution and consolidate results.
Feature Test Owner – Auth	Roopa Sreedhar A	Test authentication, roles and session controls.
Feature Test Owner – Catalog	Sakthevel Saravanan	Test product/category/search functionality.
Feature Test Owner – Stock	Samiksha B N	Test stock-in/out, integrity and reliability.
Feature Test Owner – Supplier/Reports	Rishitth G R	Test suppliers, alerts and reporting.

10.Risks and Mitigation

Risk	Impact	Mitigation
Unstable build	Blocks many tests and invalidates results.	Run smoke tests first; isolate blockers and retest after fixes.
Database/test-data inconsistency	False failures or incorrect stock results.	Reset controlled test fixtures before test runs.
Performance environment differs from production	Results may not represent production behavior.	Record test load/environment and compare repeated runs.
Security testing omitted from regression	Privilege/data-exposure defects may reappear.	Keep security cases in the regression suite and CI where practical.

11. Assumptions & Dependencies

- Django authentication and PostgreSQL are available in the test environment.
- Representative Staff and Administrator test accounts are available.
- The implemented application follows the SRS architecture and interfaces.
- No external payment, ERP or hardware integration is required for Version 1.0 testing.

12. Suspension & Resumption Criteria

Suspend testing when the test environment is unavailable, the build blocks a substantial portion of planned cases, or a data-integrity defect makes further execution unsafe.

Resume when the environment is stable, blocking defects are fixed or formally waived, and test data is restored to a known baseline.

13. Test Cases and Traceability

The following planned cases cover functional, non-functional and security requirements. Actual Result and Status are intentionally marked Not Executed because this document is a test plan, not an execution report.

Test ID	Req ID	Scenario	Precondition	Test Data / Action	Expected Result	Actual Result	Status
UT-AUTH-01	FR-001	Valid login	Valid Staff account	Correct credentials	Authenticated session and Staff dashboard	Not Executed	Not Executed
UT-AUTH-02	FR-001	Invalid login	Valid account	Wrong password	Login rejected; no session created	Not Executed	Not Executed
ST-AUTH-01	FR-002/ SEC-002	RBAC enforcement	Staff account	Open adminonly URL/action	Access denied; event logged	Not Executed	Not Executed
ST-AUTH-02	FR-004/ SEC-004	Session expiry	Authenticated user	Remain idle beyond configured period	Session expires; re-login required	Not Executed	Not Executed
UT-CAT-01	FR-005	Add product	Admin/ authorized user	Valid product fields	Product saved with unique SKU and stock	Not Executed	Not Executed
UT-CAT-02	FR-006	Edit product	Existing product	Change price/category	Updated details displayed	Not Executed	Not Executed
ST-CAT-01	FR-007	Disable product	Existing product with history	Disable product	Product unavailable for new sale; history retained	Not Executed	Not Executed
ST-CAT-02	FR-008	Search/filter	Catalog populated	Search by name/category/SKU	Matching products returned	Not Executed	Not Executed
ST-CAT-03	FR-009	Current stock display	Product with known quantity	Open catalog	Displayed quantity matches database	Not Executed	Not Executed
IT-STOCK-01	FR-010	Record stock-in	Valid supplier/product	Quantity > 0	Stock increases; transaction recorded	Not Executed	Not Executed
IT-STOCK-02	FR-011	Record stock-out	Active product with stock	Sell quantity <= stock	Stock decreases; transaction recorded	Not Executed	Not Executed
UT-STOCK-02	FR-012	Reject over-sale	Product stock=5	Attempt sale=6	Transaction rejected; stock remains 5	Not Executed	Not Executed
IT-STOCK-03	FR-013	Unique transaction ID	New stock transaction	Record transaction	Unique transaction ID generated	Not Executed	Not Executed
ST-STOCK-04	FR-014	Audit trail	Recorded stock transaction	Inspect audit record	User, timestamp and quantity present	Not Executed	Not Executed
UT-SUP-01	FR-015	Supplier CRUD	Admin authenticated	Add/edit/remove supplier	Supplier record updated as requested	Not Executed	Not Executed
IT-SUP-02	FR-016	Supplier association	Registered supplier	Record stock-in	Transaction references supplier	Not Executed	Not Executed
ST-ALERT-01	FR-017	Low-stock alert	Product threshold configured	Reduce quantity below threshold	Alert appears in Staff/Admin view	Not Executed	Not Executed
ST-REPORT-01	FR-018	Date-range report	Transactions exist	Request report for range	Aggregated stock/sales report displayed	Not Executed	Not Executed
ST-REPORT-02	FR-019	Top-selling products	Sales history exists	Open report	Top-selling items ranked from recorded sales	Not Executed	Not Executed
ST-PERF-01	NFR-001	Search response time	Normal test load	Run repeated catalog searches	At least 95% complete within 2 seconds	Not Executed	Not Executed
ST-AVAIL-01	NFR-002	Availability	Running service	Observe defined business window	Availability meets 99% excluding planned	Not Executed	Not Executed
					maintenance		
ST-USE-01	NFR-003	Transaction usability	Staff test user	Complete stockin/out	Completed in <=4 screens/steps	Not Executed	Not Executed
ST-REL-01	NFR-004	Duplicate submission	Valid stock transaction form	Double-submit/ retry	Only one transaction recorded	Not Executed	Not Executed

ST-SEC-01	SEC-001	Password storage	Test account	Inspect stored credential	Password stored as hash, not plaintext	Not Executed	Not Executed
ST-SEC-03	SEC-003	Input validation	Stock forms	Submit negative/malformed quantity	Input rejected safely	Not Executed	Not Executed
ST-SEC-05	SEC-005	Unauthenticated data access	Logged-out browser	Open protected supplier/pricing view	Access denied; no sensitive data exposed	Not Executed	Not Executed
ST-SEC-06	SEC-006	Security event logging	Test account	Generate failed login/unauthorized request	Security event logged without secrets	Not Executed	Not Executed

13.1 Requirements Traceability Matrix

Req ID	Architecture Ref	Design Ref	Test Case ID	System Test ID	Coverage
FR-001	AUTH-01	SEQ-01	UT-AUTH-01/02	ST-AUTH-01	Covered
FR-005	CATALOG-01	Product class	UT-CAT-01	ST-CAT-01	Covered
FR-010	STOCK-01	Stock service	IT-STOCK-01	ST-STOCK-01	Covered
FR-012	STOCK-02	Stock validation	UT-STOCK-02	ST-STOCK-02	Covered
FR-017	REPORT-01	SEQ-03	UT-ALERT-01	ST-ALERT-01	Covered
SEC-002	AUTH-02	RBAC middleware	ST-SEC-01	ST-AUTH-01	Covered
NFR-001	API-01	Search contract	ST-PERF-01	ST-PERF-01	Covered

14. Test Metrics & Reporting

- Planned vs executed test cases.
- Pass/fail/not-executed counts and requirement coverage.
- Defect count by severity, defect aging and reopen rate.
- Performance percentile for catalog search.
- Security test coverage for SEC-001 through SEC-006.
- Regression coverage after fixes.

Reports: execution log, defect report, performance/security evidence and final Test Summary Report.

15. Approvals

Role	Name	Signature / Date
Team Representative	Team 8	
Faculty / Reviewer		
Final Sign-off		